Microservices

Microservicios

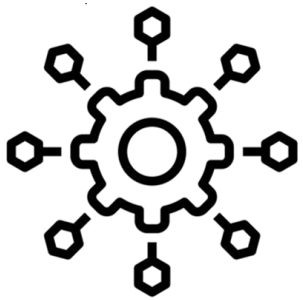
Universidad de La Sabana

FACULTAD DE INGENIERÍA

Universidad de La Sabana

FACULTAD DE INGENIERIA

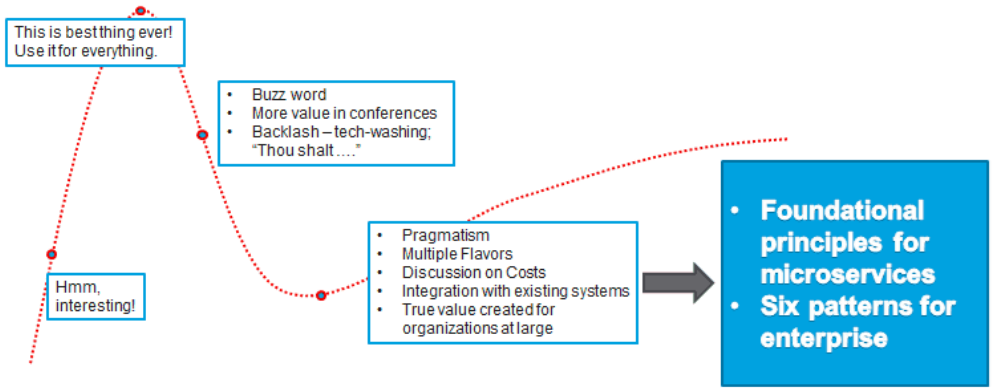
Agenda

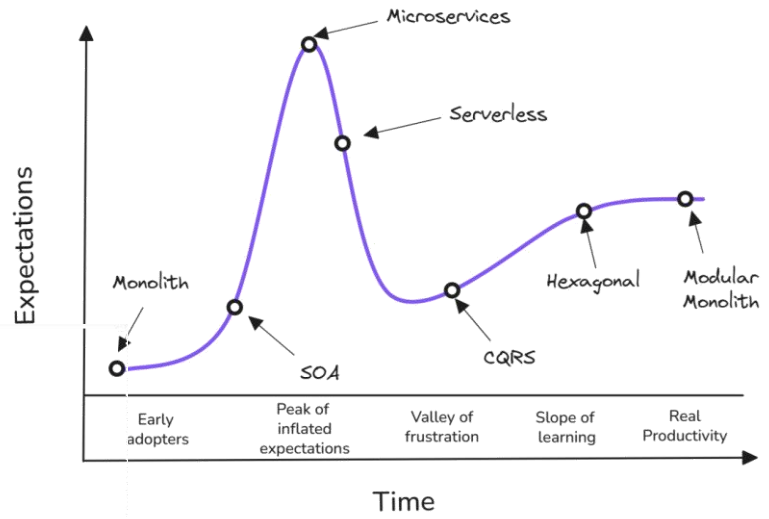


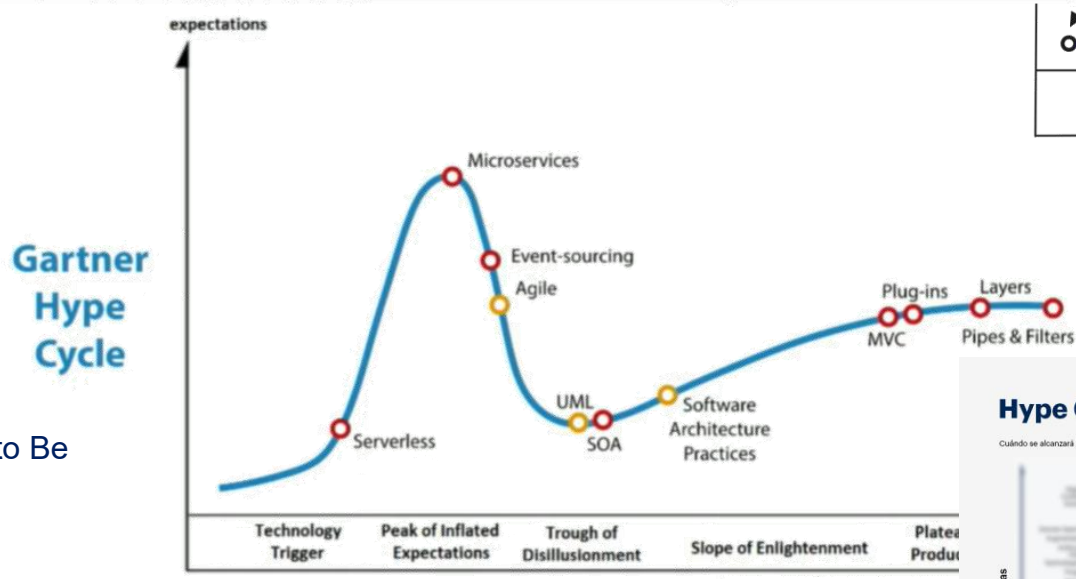
Universidad de La Sabana
FACULTAD DE INGENIERIA

Software Architecture Hype Cycle

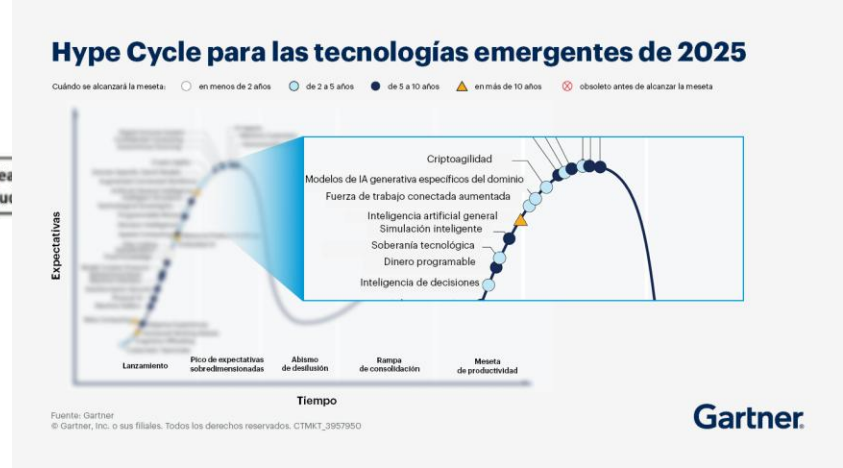
"Coolness factor" of emergent technologies







TechWorld WithMilan



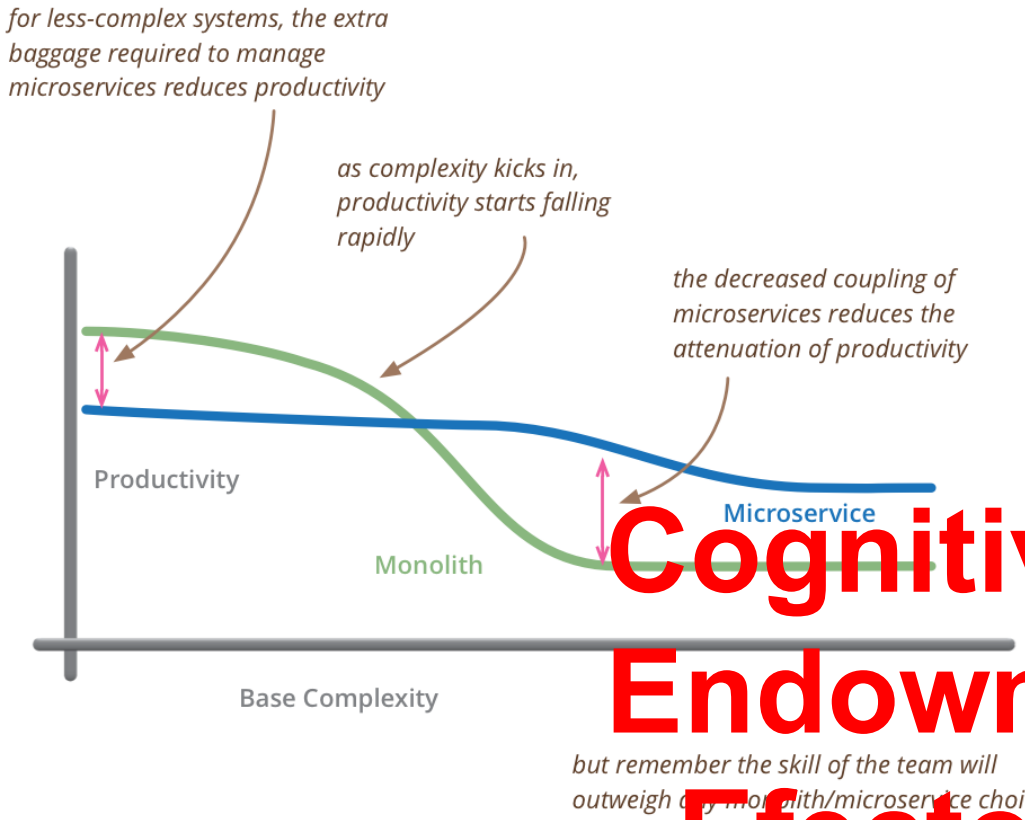
[2007] Mulesoft Blog. Are Microservices Too Cool to Be Useful?. Sandeep Singh Kohli
[2018] NDC Conf. Deliberate Architecture. Robert Smallshire
[2026] TechWorld with Milan, Software Arch Hype Cycle. Dr. Milan Milanovic

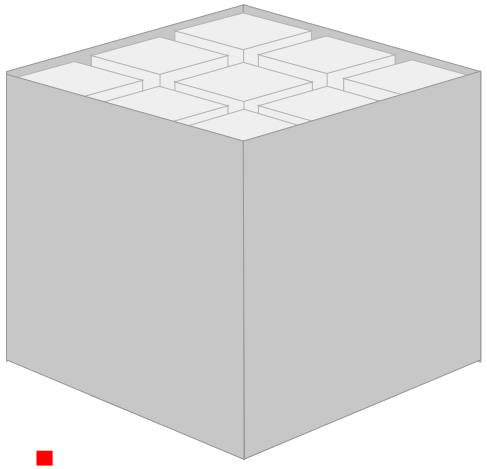
Example from
[2025] Gartner, Emergent Technologies on 2025.

Link

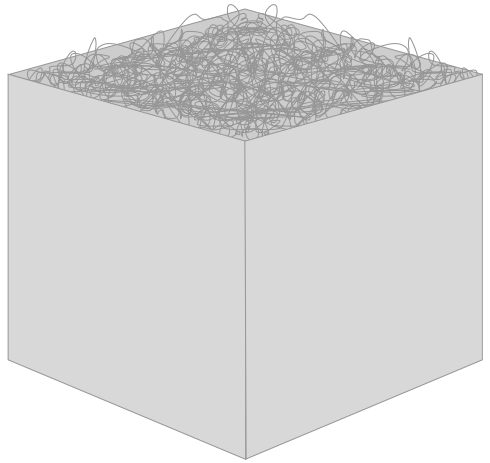
Universidad de La Sabana

What is it mean ?





Hope vs. Reality



Cognitive bias: Endowment Effect - Efecto de Dotación

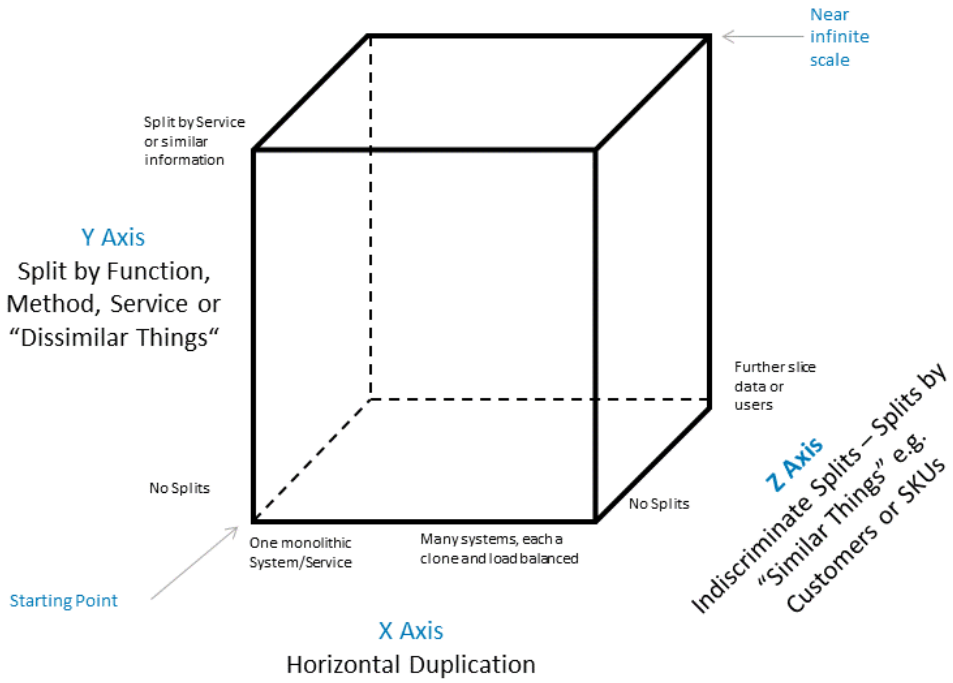
- I hope you aren't a presales engineer: goal on sales or specific services
- Your focus is real productivity and reduced time-to-market
- What are the factors to choose?: tech (requisites, current products), non-tech (team maturity, management, egoes)

- Greenfield vs Brownfield projects: Starting point.
- Monoliths breed irreversible tight coupling
- Hard boundaries protect system architecture
- The "refactoring shift" (Small vs. Large)

Links for MartinFowler.com

Scale Cube

AKF Scale Cube

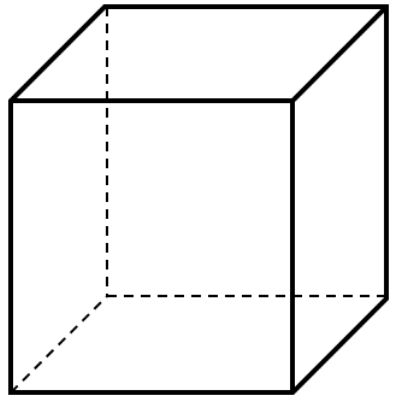


AKF Scale Cube

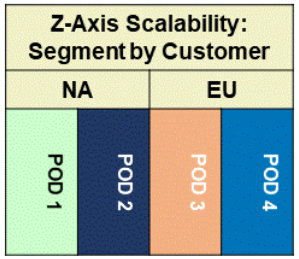
AKF Scale Cube

AKF Scale Cube

Pros	
Transaction Scalability	Scales transactions well
Latency	Reduces response times
Cache	Increases cache hit rate
Availability	Can have fault isolation



Cons	
Difficult	Can take time to architect
Complexity	More things to manage

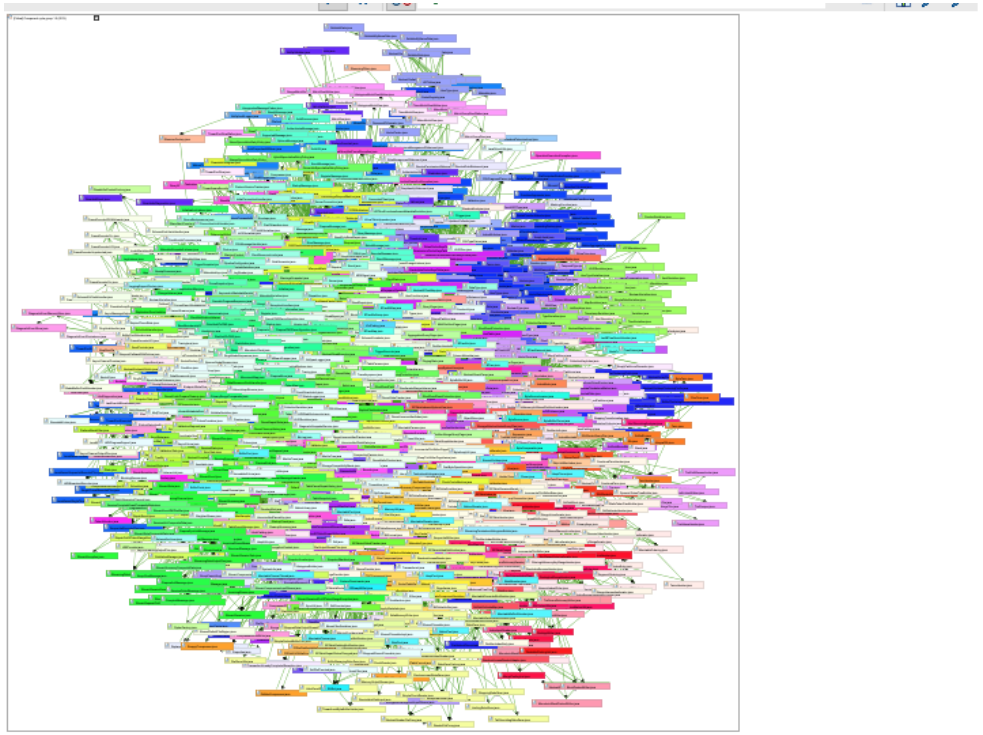


X: Redundancy/HA → Previous subject (Cloud Services)
Y: Divide application → Microservices (This module)
Z: Divide data → Distributed Services (Next module)

In addition, you can find original at [AFK Partners]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Evolution from monolith



Properties | Parser Depende... | Parser Depende... | Markers | [Automated] ./C... | '[Critical] Comp...

Name [2,013]	Incoming	Outgoing	Incoming Interfac	Interface Score	Coupling Scor
ColumnFamilyStore.java	201	127	86	10,922	25,527
TableMetadata.java	367	48	109	5,232	17,616
DatabaseDescriptor.java	304	46	1	46	13,984
StorageService.java	76	169	1	169	12,844
SSTableReader.java	155	62	77	4,774	9,610
FBUtilities.java	245	34	0	0	8,330
ClusterMetadata.java	180	45	15	675	8,100
Verb.java	61	86	0	0	5,246
AbstractType.java	273	18	232	4,176	4,914
Keyspace.java	108	42	31	1,302	4,536
MessagingService.java	119	36	1	36	4,284
ColumnMetadata.java	152	28	62	1,736	4,256
ReadCommand.java	53	79	48	3,792	4,187
DecoratedKey.java	279	15	264	3,960	4,185
SystemKeyspace.java	58	71	0	0	4,118
PartitionUpdate.java	70	53	47	2,491	3,710
CassandraMetricsRegistry.java	61	55	1	55	3,355
ClientState.java	122	27	98	2,646	3,294
ActiveRepairService.java	37	77	3	231	2,849

Why Should a Company Decide to Build an Evolutionary Architecture?

Predictable versus evolvable: The Innovators Dilemma
(companies in well-established markets are likely to fail as more agile startups address the changing ecosystem better).
Scale: No more Band-Aids, focus on decoupling
Advanced business capabilities: (A/B Testing) → As variation, Cycle time as a business metric
Adaptation versus evolution: Adaptation with Tech Debt on Feature Toggles vs Fundamental Change

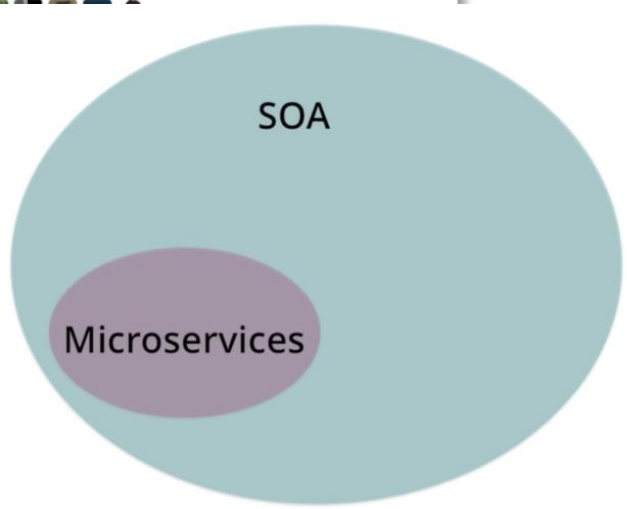
Why Should a Company Decide Not to Build an Evolutionary Architecture?

Can't evolve a Big Ball of Mud: No feasibility
Other architectural characteristics dominate: Other QA instead of evolutionarily
Sacrificial architecture: MVP
Planning on closing the business soon

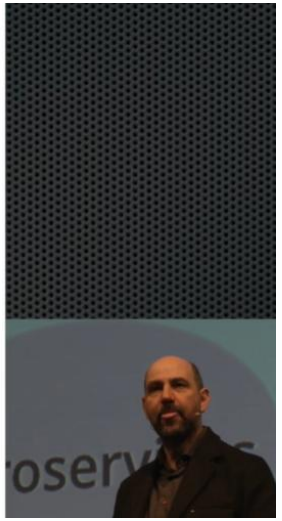
[Link] and [BEAr, Ch9]

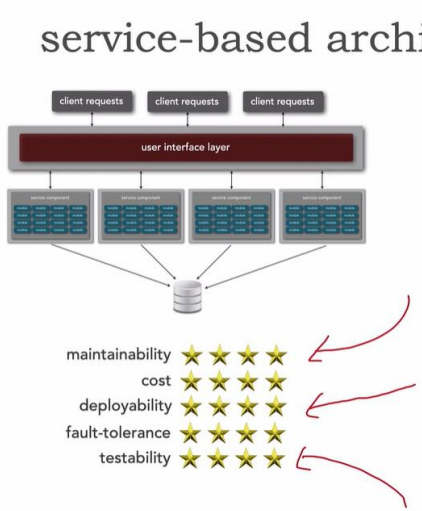
Universidad de La Sabana
FACULTAD DE INGENIERIA

SOA, SBA, MSA, Functions and so on



Dimension	Service-Oriented (SOA)	Service-Based (SBA)	Microservices Arch (MSA)
Primary Scope	Enterprise Integration	App-level Modularity	Fine-grained Agility & Scale
Service Size	Coarse (Large Macro-services)	Medium (Domain Modules)	Fine (Single Responsibility)
Database Strategy	Shared Enterprise Database	Commonly Shared Instance	strictly Database-per-Service
Integration Middleware	Centralized ESB (Smart Pipes)	Lightweight Gateway/Broker	Decoupled (Dumb Pipes)
Deployment Dependency	High Coordinated Coupling	Moderate	Completely Autonomous





when to use...

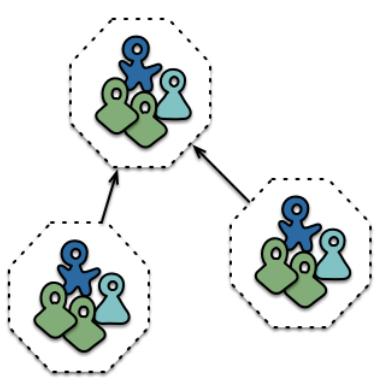
See sidebar on [MFMs] too

Links

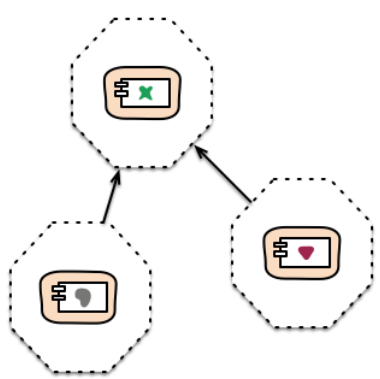
Universidad de La Sabana
FACULTAD DE INGENIERIA

Features

Componentization via Services
Organized around Business Capabilities
Products not Projects
Smart endpoints and dumb pipes
Decentralized Governance
Decentralized Data Management
Infrastructure Automation
Design for failure
Evolutionary Design

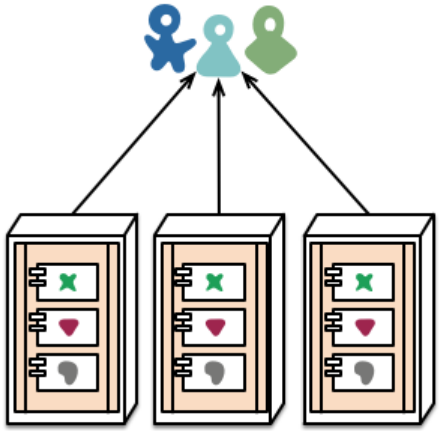


Cross-functional teams...

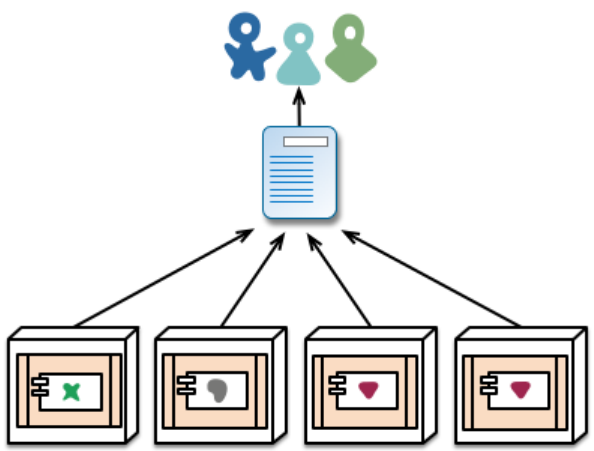


... organised around capabilities
Because Conway's Law

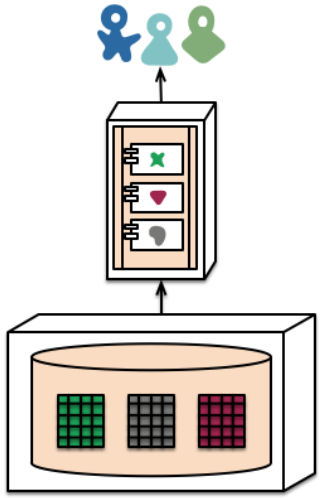
Divided:
- Technical
- Organizational
- Architectural



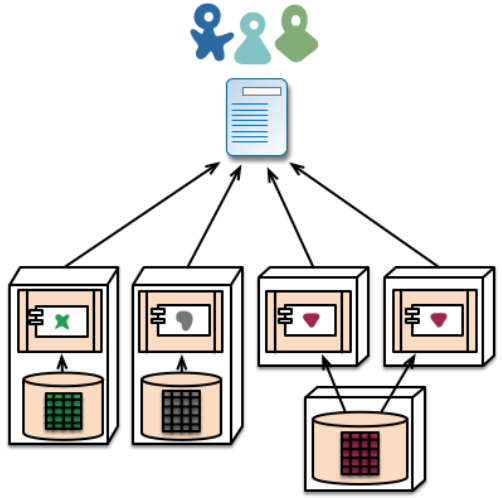
monolith - multiple modules in the same process



microservices - modules running in different processes



monolith - single database



microservices - application databases

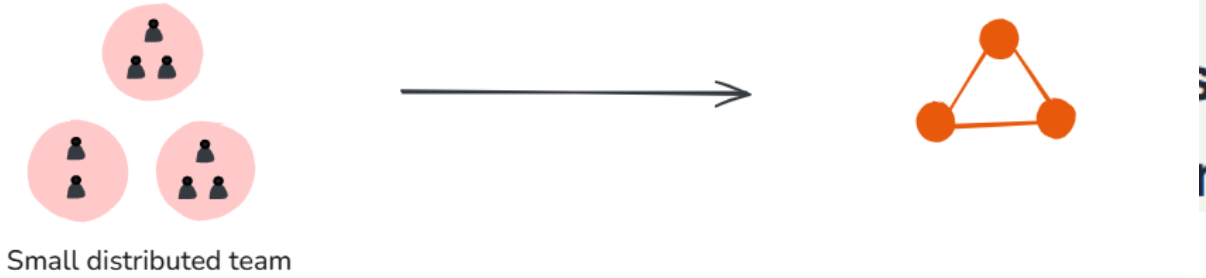
[MFMs]

Universidad de La Sabana
FACULTAD DE INGENIERIA

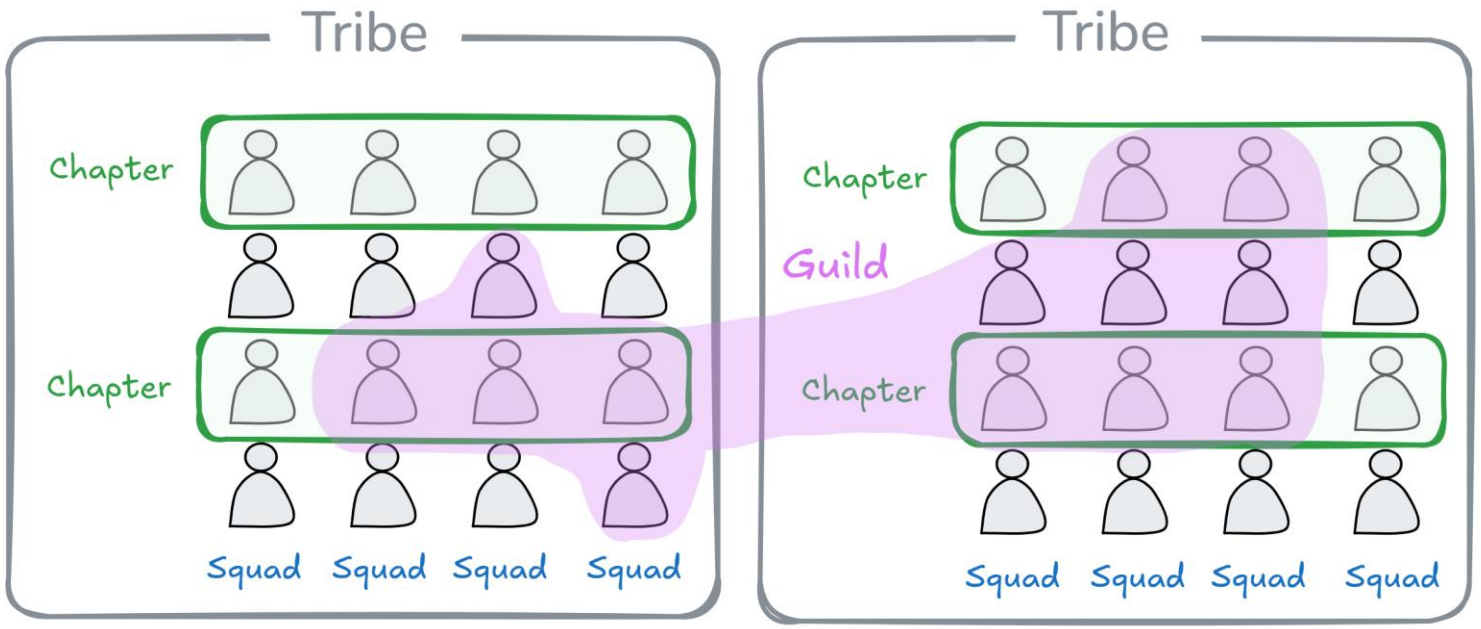
Conway's Law

Laws of Software Engineering

NEWSLETTER INFO BOOK POSTER

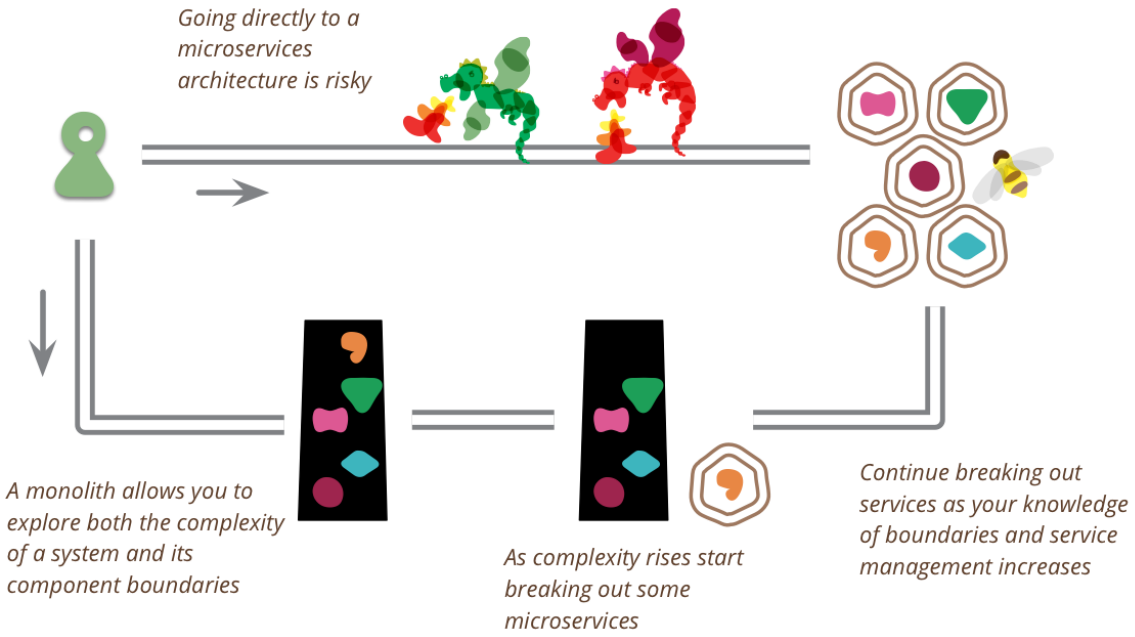


s that shape
ns.



Universidad de La Sabana
FACULTAD DE INGENIERIA

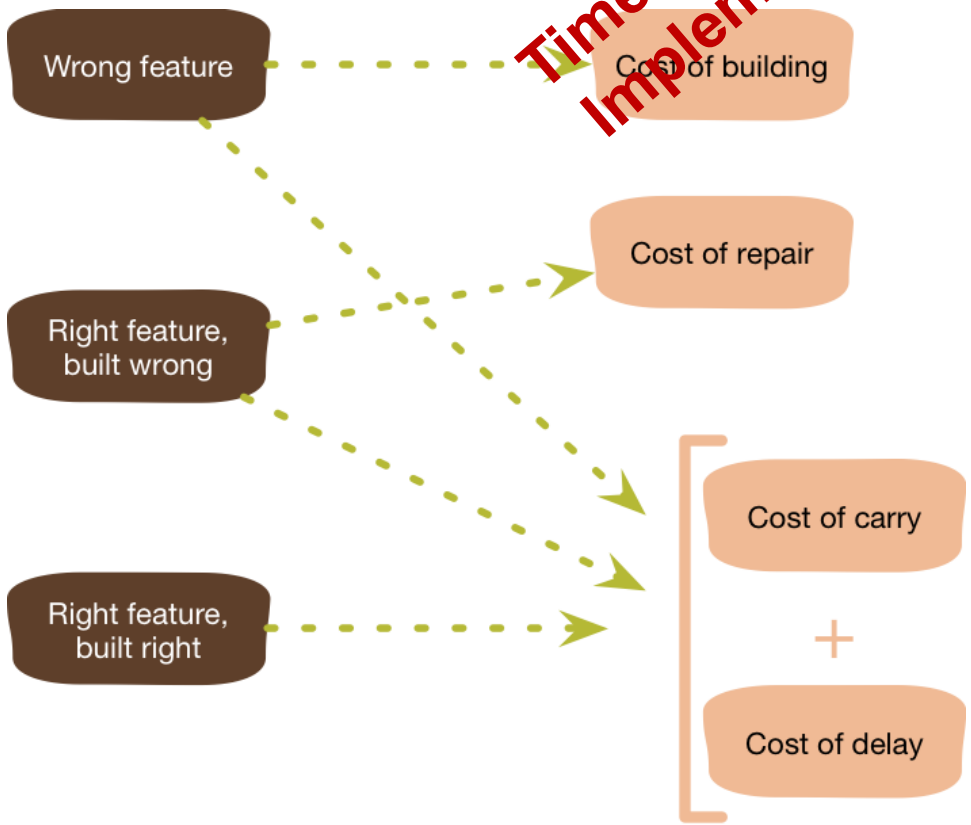
Monolith-First



Squad/Team/Tribe/Cell Maturity and Well-Design Thinking

- YAGNI

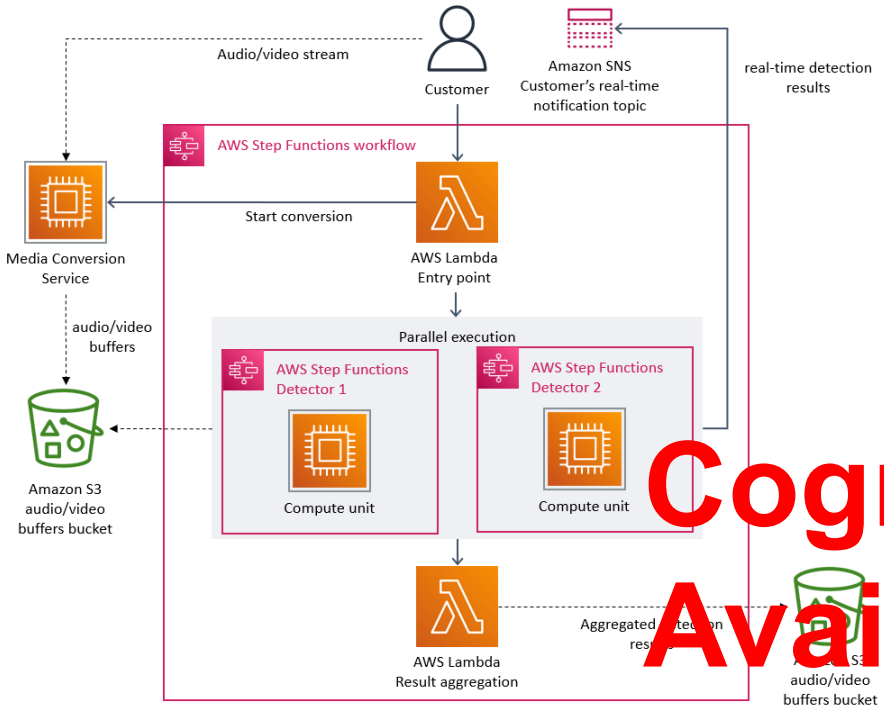
You Aren't Gonna Need It

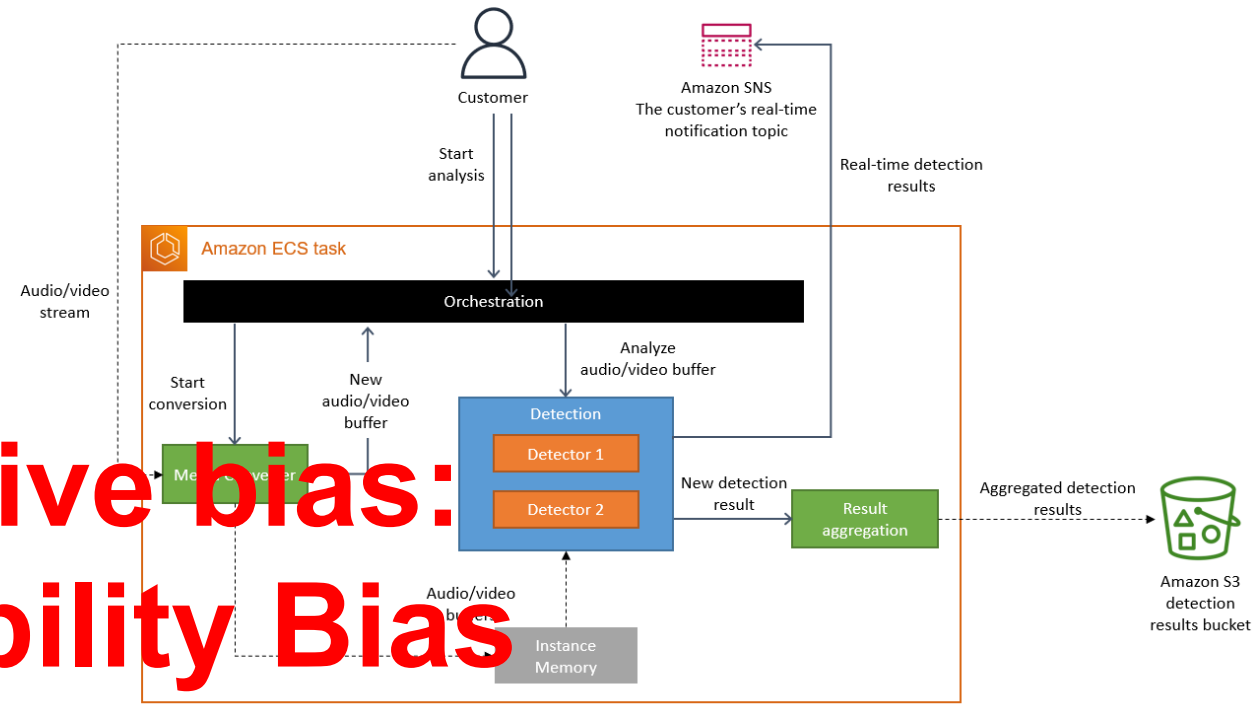


Golden bullet ? → Capacity and Maturity
Req. Timeline (Market Change?)
Dummy interfaces (API GW, LB) → Cost of Carry
API Design →Business Expert

Universidad de La Sabana
FACULTAD DE INGENIERIA

Clever example of overarchitecting





Cognitive bias: Availability Bias - Sesgo de disponibilidad

Problem:
Review defect after Media Conversion
Initial Review:

- Orchestrate using Step Functions
- Lambda concurrency
- S3 Exchange

>> Initial MVP

Problem:
Review defect after Media Conversion
Initial Review:

- Orchestrate using Step Functions
- Lambda concurrency

>> Initial MVP

Universidad de La Sabana

FACULTAD DE INGENIERIA

Ubiquotous Language

Universidad de La Sabana
FACULTAD DE INGENIERIA

Iteration of Bound Context

Universidad de La Sabana

FACULTAD DE INGENIERIA

Containers and Serverless

Universidad de La Sabana
FACULTAD DE INGENIERIA

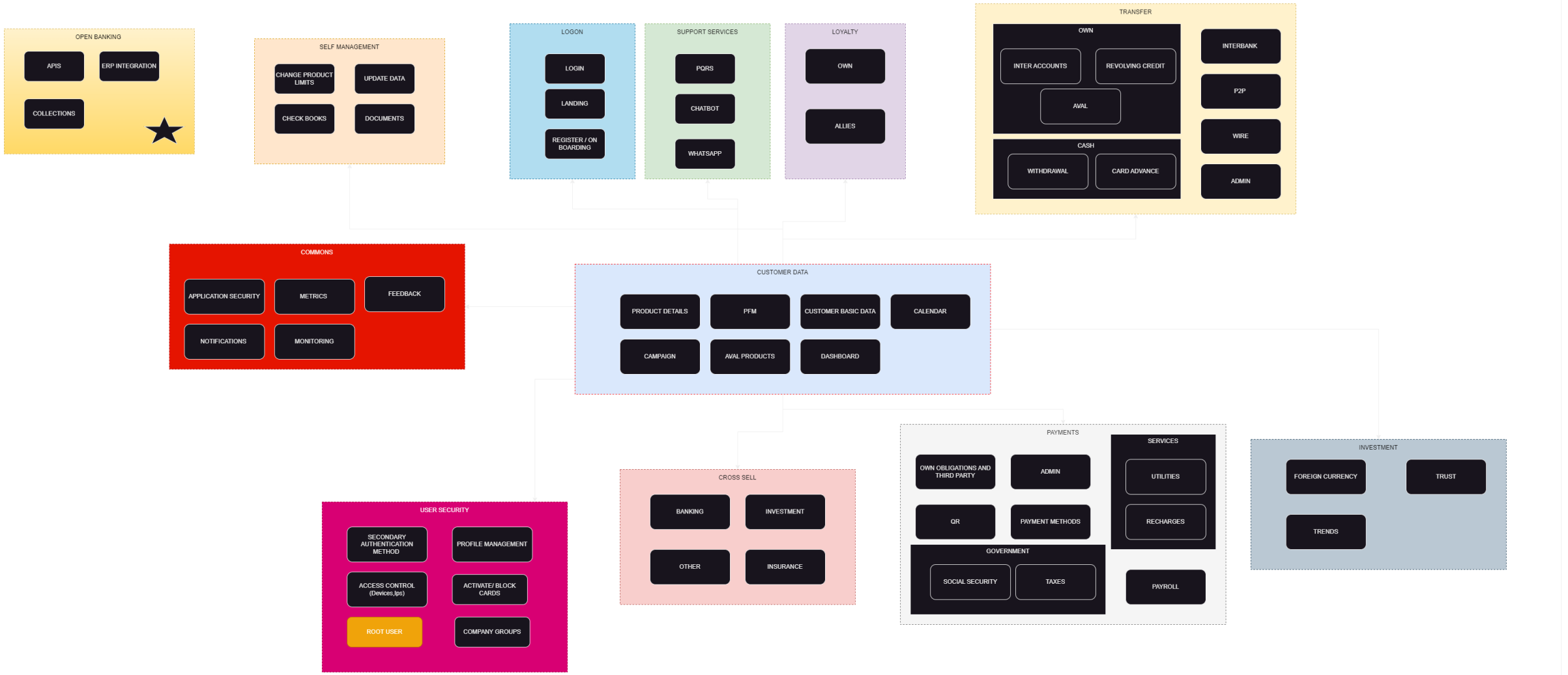
Related Cloud Services

Example using API Gateway

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Study Cases



Owner: Banco de Bogotá
Project: Portal BDB Empresas
Diagram: Domain Driven Design
Authors: Mauricio Castro / Sebastian Romero

Universidad de La Sabana

FACULTAD DE INGENIERIA

eCommerce

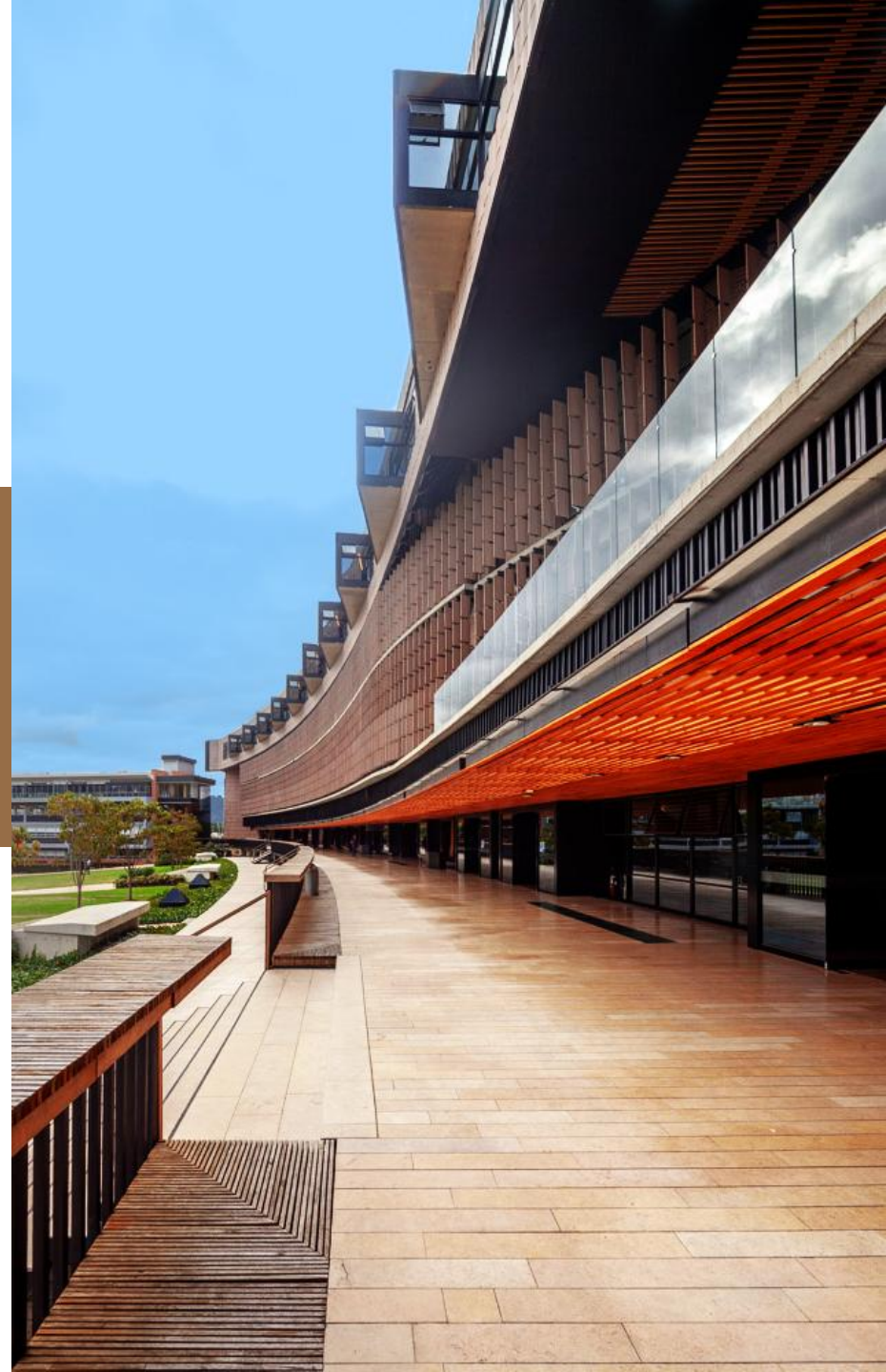
Universidad de La Sabana

Design Techniques: DDD

Consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet.Consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet.

Universidad de La Sabana

FACULTAD DE INGENIERÍA



Universidad de La Sabana
FACULTAD DE INGENIERIA

Strategical and Tactical Analysis

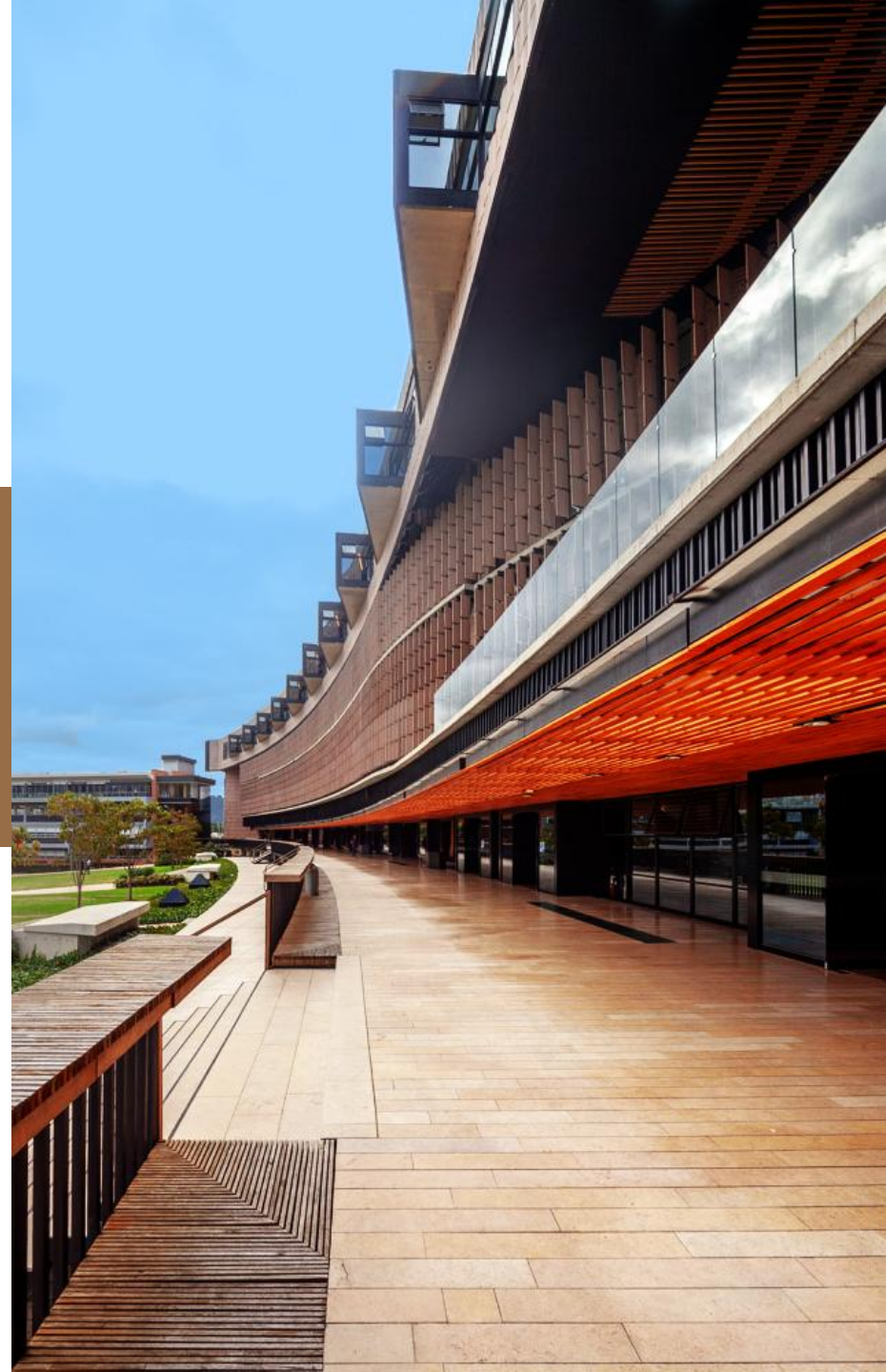
Universidad de La Sabana

Design Techniques: Assemblage

Change the name after of it. Richardson.

Universidad de La Sabana

FACULTAD DE INGENIERÍA



Universidad de La Sabana
FACULTAD DE INGENIERIA

Strategical and Tactical Analysis

Strangler Fig

Consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet.Consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet.

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Universidad de La Sabana
FACULTAD DE INGENIERIA

Strangler Fig and Anticorruption Layer

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Evolution of Microservices

Example using Microfrontends - MFE

Universidad de La Sabana
FACULTAD DE INGENIERIA

Strangler Fig and Anticorruption Layer

Universidad de La Sabana
FACULTAD DE INGENIERIA

References

- [BEAr] Ford, N., Parsons, et all (2022). Building evolutionary architectures: Support constant change, 2nd ed. O'Reilly Media, Inc.
  Ch. 1, 26 & 27

- [SAIP]Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.
  Ch. 1, {QA} 4 to 14, {ASR} 19, {ADD} 20

- [APRe] Richards, M. (2015). Software architecture patterns (Report). O'Reilly Media.
  Ch 1, {Arch Pattern} Ch. 3 to 7

- [AzDE] Microsoft. (n.d.). Architecture styles. Azure Architecture Center.

Universidad de La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co